OUTPUT ANALYSIS

Hospital Patient Management System • Circular Doubly Linked List, Array and Queue in C++

1. Time Complexity Analysis

Let **n** be the number of patient records currently stored in the circular doubly linked list.

Operation	Time Complexity	Reason
Admit Patient	**O(n)**	Duplicate-ID check scans the list; finding a free bed is a fixed 10-bed scan; insertion at tail (*head->prev*) is O(1).
Discharge Patient	**O(n)**	Searching the ID takes O(n); enqueue (with a dequeue if the queue is full), freeing the bed and unlinking the node are O(1).
Search Patient	**O(n)**	Linear traversal from *head* until the head is reached again; a match on ID may need every node.
Display Forward / Backward	**O(n)**	Walks the circular list once using *next* from head or *prev* from tail until it returns to the start, printing each node.
Bed Status	**O(1)**	Scans the fixed-size bed array (MAX_BEDS = 10), independent of the number of patients.
Undo Discharge (Re-admit Oldest)	**O(n)**	Peek/dequeue at the queue front is O(1); duplicate-ID check is O(n); re-insertion at tail is O(1).
Discharge History	**O(1)**	Reads at most QSIZE = 5 records from front to rear of the queue, so it is bounded by a constant.

2. Space Complexity Analysis

Patient data is stored in dynamically allocated nodes. Each node holds the patient fields (ID, name, age, disease, bed number) and two pointers: *prev* and *next*.

Total Space Complexity	**O(n)**	The list holds n patient nodes (fields plus *prev* and *next*); the bed array and queue are fixed-size.
Bed Array	**O(1)**	Always MAX_BEDS = 10 elements, so it does not grow with the number of patients.
Discharge Queue	**O(1)**	Circular array queue of QSIZE = 5 records; the size is fixed at compile time and the indices wrap around using modulo.
Auxiliary Space per Operation	**O(1)**	Operations use a few temporary pointers and variables; admitting a patient allocates one extra node.

Interpretation: Operations that must locate a patient (admit, discharge, search, undo) are linear-time because the circular list is searched from *head*. Reaching the tail through *head->prev* makes insertion O(1), and a doubly linked list makes unlinking O(1) once the node is found. The circular queue enqueues and dequeues in O(1) with no shifting, and bed allocation and discharge history are bounded by fixed array sizes, so they behave as constant time.